MiniShop - Note de cybersécurité

> **Objet** : failles testées, corrections apportées et code correspondant.

> **Périmètre** : application MiniShop livrée (PHP 8.2, moteur de données JSON, zéro dépendance runtime), d'après le plan d'attaques S-01...S-10 du document de tests (`docs/02`, §5) et les exigences SEC-01...SEC-13 du cahier des charges (`docs/01`, §8).

> **Date** : 28/09/2026 - **Rejouable** : `npm run test:all` (toutes les attaques ci-dessous sont rejouées en continu par la suite E2E - aucune n'est vérifiée « à la main »).

---

1. Stratégie : défense en profondeur, preuve par l'attaque

La sécurité de MiniShop repose sur **quatre lignes de défense indépendantes**, et chaque ligne est attaquée par la suite E2E (`scripts/wasm-e2e.mjs`, batteries T6, T12, T13 - 155 vérifications) :

Ligne	Rôle	Où dans le code
1. Validation & moteur typé	toute donnée entrante traverse des accesseurs typés bornés, puis un moteur à contraintes (types, ENUM, CHECK, UNIQUE, FK)	`app/Controller/Controller.php` (`int()`, `str()`, `float()`, `bool()`), `app/Model/Data/Schema.php`, `JsonStore.php`
2. Échappement systématique en sortie	aucune vue n'affiche une donnée sans `e()`	`app/Security/xss.php`, toutes les vues `app/View/**`
3. Authentification & autorisation	sessions durcies, jetons CSRF, gardes de rôle par route, contrôle d'appartenance	`app/Security/Auth.php`, `Csrf.php`, `app/Router.php`, contrôleurs
4. Durcissement HTTP & erreurs	en-têtes défensifs, routeur à liste blanche, erreurs génériques, journal d'audit	`public/index.php`, `app/Model/Data/Journal.php`

Principe directeur : la sécurité n'est pas documentée, elle est **exécutée**. Chaque faille ci-dessous est accompagnée du test qui la rejoue - une régression est donc détectée par `npm run test:all`, pas par une relecture.

---

2. Fondation structurante : il n'y a rien à injecter (S-01)

Avant même de parler de filtrage : **l'application n'embarque aucun interpréteur de requêtes**. La couche de données (§ contrainte de projet : « pas de SQL ») est un moteur JSON dont les requêtes sont construites en PHP, jamais écrites en texte.

```
// app/Model/Data/Filter.php - une condition est un triplet PHP typé, pas une chaîne
public static function eq(string $col, mixed $value): array
{
    return [$col, '=', $value];
}
```

```
// app/Model/Data/JsonStore.php - les opérateurs sont évalués dans un
// switch à liste blanche ; une valeur ne peut JAMAIS devenir du code :
private function matchesOne(array $row, array $condition): bool
{
    [$col, $op, $value] = [$condition[0], strtolower($condition[1]), $condition[2] ?? null];
    $cell = $row[$col] ?? null;
    switch ($op) {
        case '=':  return $this->looseEquals($cell, $value);
        case '!=': return !$this->looseEquals($cell, $value);
        // ... '<', '<=', '>', '>=', 'like', 'in', 'between', 'null' ...
        default:
            throw new QueryError("Opérateur de filtre inconnu : « {$op} »");
    }
}
```

Trois conséquences de sécurité :

1. **Aucune concaténation/interpolation de requête** : `' OR 1=1 --` est traité comme un mot-clé de recherche ordinaire (recherche pliée insensible casse/accents), pas comme du langage.
2. **Garde-fou structurel** : une condition mal formée (clés non positionnelles - bug réel découvert par la campagne de tests et corrigé) lève `QueryError` au lieu de matcher silencieusement :

```
// app/Model/Data/JsonStore.php::matches() - durcissement ajouté après découverte
if (!array_is_list($condition)) {
    throw new QueryError('Condition mal formée (liste [colonne, opérateur, valeur] attendue) : ' .
 var_export($condition, true));
}
```

1. **Contraintes de schéma en profondeur** : même une donnée malveillante qui franchit les contrôleurs est confrontée au moteur (NOT NULL, types, ENUM de statuts, CHECK `prix_ht > 0` / `stock >= 0`, UNIQUE référence/email/slug, FK RESTRICT/CASCADE) - cf. `tests/php/engine_smoke.php`, vert.

Preuves (E2E T12) : `q=' OR 1=1 --` -> 200, « 0 produit trouvé », la table `client` intacte ; `q=%'; DROP TABLE produit;--` -> 200, aucune donnée perdue. **Tests fonctionnels** : tri inconnu (`?tri=prix_desc;DROP TABLE produit`) -> repli sur la liste blanche `TRIS = ['nom','prix_asc','prix_desc','nouveaute','stock']` sans erreur.

---

3. Les 10 attaques du plan de validation - détail par faille

S-01 - Injection (SQL/requête) - [x] neutralisée par construction

Risque (OWASP A03:2021)	un payload dans un paramètre modifie la sémantique d'une requête pour lire/écrire la base

Attaque testée	E2E T12 : `GET /recherche?q=' OR 1=1 --` et `GET /recherche?q=%'; DROP TABLE produit;--` ; catalogue : `?tri=prix_desc;DROP TABLE produit`
Défense	§2 ci-dessus : moteur JSON sans interpréteur, opérateurs en liste blanche, contraintes de schéma ; en aval, `filter_var(FILTER_VALIDATE_EMAIL)` sur les emails, `max`/`step` HTML complétés par les bornes serveur (`str('email', 190)`, etc.)
Preuve	E2E T12 (2 vérifications) + `features_test.php` « tri inconnu -> repli liste blanche » + `run_tests.php` T-25

S-02 - XSS stocké - [x] échappement systématique + CSP

Risque (OWASP A03)	un script injecté en base est exécuté dans le navigateur d'autres utilisateurs (vol de cookie/session)
Attaque testée	E2E T12 : l'admin crée un produit nommé `<script>alert("xss")</script> Bombe`, puis un visiteur ouvre `GET /recherche?q=Bombe`
Défense	échappement **à la sortie** uniquement (jamais à l'entrée - on conserve la donnée originale), via un unique point de passage :

```
// app/Security/xss.php - chargé par bootstrap, utilisé par TOUTES les vues
function e(mixed $valeur): string
{
    return htmlspecialchars((string) ($valeur ?? ''), ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

En profondeur : `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'; base-uri 'none'; form-action 'self'` (`public/index.php`) - même un échappement manqué ne s'exécuterait pas.

Preuve	E2E T12 : la page contient `<script>` (littéral) et **aucune** occurrence de `<script>alert`

S-03 - IDOR (accès aux ressources d'autrui) - [x] contrôle d'appartenance systématique

Risque (OWASP A01)	`GET /mes-commandes/42` d'un autre client livre ses données
Attaque testée	E2E T12 : le client *bruno* boucle sur `/mes-commandes/1..4` ; E2E T6 : accès direct à une commande d'autrui ; fonctionnels : `cancelOrder` sur la commande d'un autre
Défense	l'identifiant de la requête n'est **jamais** suffisant : il est confronté à la session côté repository

```
// app/Controller/CommandeController.php + CommandeRepository.php
if ($idClient !== null && (int) $commande['id_client'] !== $idClient) {
    throw new BusinessError('ACCES_NON_AUTORISE');
}
```

Preuve	E2E T12 « boucle IDOR 1..4 : toujours redirigé » + T6 + `features_test.php` (refus `ACCES_NON_AUTORISE` sur annulation/validation d'autrui)

S-04 - Falsification du prix depuis le navigateur - [x] le serveur fait foi

Risque	ajouter `prix_unitaire: 0.01` au POST du panier pour acheter à 1 centime
Attaque testée	E2E T9 : `POST /panier/ajouter { id_produit: 5, quantite: 1, prix_unitaire: '0.01' }`
Défense	le panier ne stocke que `id_produit -> quantité` ; **tout montant est recalculé en relisant le produit** côté serveur, et figé à l'achat par le déclencheur (RB-06) :

```
// app/Model/PanierSession.php::recapitulatif() - le prix vient du magasin, jamais du client
$produit = $this->store->find('produit', $idProduit);
$prixTtc = round((float) $produit['prix_ht'] * (1 + (float) $produit['tva'] / 100), 2);
```

Preuve	E2E T9 : « prix serveur 154,80, pas 0,01 » ; `run_tests.php` (prix TTC figé, RB-06)

S-05 - Vol / fixation de session - [x] régénération à chaque élévation de privilège

Risque	imposer un identifiant de session connu avant la connexion (fixation), ou rejouer un cookie volé
Attaque testée	E2E T12 : un « pirate » impose son cookie `MINISHOPSESS=sid-pirate-fixe-123` puis se connecte en Alice

| **Défense** | cookie durci + identifiant **régénéré** à la connexion (l'ancien est détruit) + mode strict :

```
// app/Security/Auth.php
session_set_cookie_params([
    'httponly' => true,   // inaccessible en JavaScript (atténue le vol par XSS)
    'samesite' => 'Lax',  // le cookie n'est pas envoyé depuis un site tiers (atténue CSRF)
    'secure'   => (($_SERVER['HTTPS'] ?? '') !== '' && ($_SERVER['HTTPS'] ?? 'off') !== 'off'),
]);
session_name('MINISHOPSESS');
ini_set('session.use_strict_mode', '1'); // identifiant inconnu ⇒ nouvelle session

public static function connecterClient(int $idClient): void
{
    self::regenererId(); // anti-fixation (SEC-05, S-05) - l'ancien sid est invalidé
    $_SESSION['client_id'] = $idClient;
}
```

En WASM (bac à sable uniquement), `regenererId()` détruit le fichier du pool et tire un nouveau

sid (`bin2hex(random_bytes(20))`) - comportement équivalent au `session_regenerate_id(true)` natif.

Preuve	E2E T12 : « session régénérée (anti-fixation) » (le sid final != sid imposé) et « le panier volé est vide » (nouvelle session)

S-06 - CSRF sur une action mutative - [x] jeton obligatoire, comparaison constante

Risque	un site tiers fait poster un formulaire authentifié à l'insu du client (ajout au panier, validation de commande...)
Attaque testée	E2E T6 : `POST /connexion` sans jeton `_csrf`

| **Défense** | **aucune** route POST n'échappe à `verifierCsrf()` (contrôleur de base) ; jeton par session de 64 hex (`random_bytes(32)`), comparé en temps constant :

```
// app/Security/Csrf.php
public static function verifier(): void
{
    Auth::demarrer();
    $recu = (string) ($_POST['_csrf'] ?? '');
    if ($recu === '' || !hash_equals(self::jeton(), $recu)) {
        http_response_code(403);
        $_SESSION['flash_erreur'] = 'Jeton de sécurité invalide ou expiré (CSRF). Rechargez la
page et réessayez.';
        Auth::rediriger('/');
    }
}
```

L'atténuation est doublée par `SameSite=Lax` sur le cookie (S-05) - un POST inter-site n'emporte même pas la session.

Preuve	E2E T6 « POST sans CSRF rejeté » ; la totalité des ~30 formulaires de l'app exige un jeton (chaque test E2E le extrait d'abord) ; `unit_tests.php` (section Csrf)
Divergence assumée	le document de tests annonçait un refus **419** (code Laravel) ; nous renvoyons **403 Forbidden**, le code standard HTTP pour une autorisation refusée

S-07 - Énumération et exposition de surface - [x] liste blanche, racine unique, en-têtes durcis

Risque	atteindre des fichiers hors périmètre (`.git`, sources SQL, config, logs), être encadré (clickjacking), ou lire des en-têtes bavards

Attaques testées	E2E T12 : `/../etc/passwd`, `/produit/../../etc/passwd`, `/%2e%2e/%2e%2e2e/etc/passwd`, `/.git/config`, `/sql/01_minishop_schema.sql`, `/app/Config/env.php`, `/var/log/application.log`, `/phpinfo.php` -> **tous 404** ; vérification des 4 en-têtes défensifs
Défense	

```
// public/index.php - en-têtes sur TOUTE réponse
header('X-Content-Type-Options: nosniff');
header('X-Frame-Options: DENY');
header('Referrer-Policy: same-origin');
header("Content-Security-Policy: default-src 'none'; frame-ancestors 'none'; base-uri 'none';
form-action 'self'");
header('X-XSS-Protection: 0'); // filtre hérité trompeur : désactivé, on s'appuie sur
l'échappement
```

```
// app/Router.php - liste blanche EXPLICITE : aucune route dynamique, aucun
// catch-all, aucune inclusion de fichier pilotée par la requête (anti-LFI/RFI)
foreach ($routes as $motif => $route) {
    $regex = '#^' . preg_replace('#\{([a-z_]+)\}#', '(?P<$1>[^/]+)', $motif) . '$#u';
    if (preg_match($regex, $chemin, $m) === 1) { /* ... */ }
}
http_response_code(404); // rien n'a matché -> 404, et c'est le SEUL comportement
```

Le contrôleur frontal vit seul dans `public/` (DocumentRoot) ; `public/.htaccess` ajoute en défense en profondeur `Require all denied` sur `.env*` et `*.log|*.sql|*.md` si jamais des fichiers sensibles étaient copiés là par erreur. Aucun `phpinfo()` dans le code (grep = 0). Pas d'upload de fichiers : les images sont des URL stockées en base (la validation du type/taille d'upload du CDC §SEC-11 n'a donc pas de surface à défendre ici).

Deux **outils de développement** assumés vivent aussi dans `public/` (`gen_volumes.php`, `mesurer.php` - ENF-01/02, conçus ainsi par le CDC §.htaccess) : servis en environnement de développement, ils **se bloquent eux-mêmes en production** (détection `APP_ENV`/env.php -> 404 volontaire, « pas 403, pour ne pas révéler l'existence de l'outil »). Ils n'ont accès à aucune table du moteur applicatif : le générateur cible la base SQL de référence (harness de volumétrie), jamais `data/minishop`.

Divergence assumée	le bac à sable PHP-WASM émet `x-powered-by: PHP/8.2.0-dev` (signature du runtime de **développement**, pas de l'application). En production Apache/FPM, l'effacement relève de la config serveur (`expose_php=Off`, `ServerTokens Prod`) - consigné dans la roadmap §6

S-08 - Fuite d'information dans les messages d'erreur - [x] messages génériques, détails côté serveur

Risque	une erreur verbeuse révèle schéma, chemins, requêtes ou existence de comptes (aide à l'attaque)

Attaques testées	E2E T2/T8 : mauvais mot de passe, email inconnu, compte bloqué, ancien mot de passe incorrect ; erreurs métier (RB-02 prix invalide, STOCK_INSUFFISANT, transitions interdites) affichées sur toutes les batteries
Défense	deux niveaux : les erreurs **métier** ne portent qu'un code + un libellé humain ; les erreurs **techniques** n'affichent rien du tout :

```
// public/index.php - filet de sécurité global
catch (BusinessError $e) {
    $_SESSION['flash_erreur'] = $e->messageHumain(); // « Aucun stock suffisant... » - aucun SQL,
aucun chemin
    Auth::rediriger($_SERVER['HTTP_REFERER'] ?? '/');
} catch (Throwable $e) {
    http_response_code(500);
    error_log('[MiniShop] ' . $e->getMessage() . ' @ ' . $e->getFile() . ':' . $e->getLine()); //
serveur seulement
    include __DIR__ . '/../app/View/errors/500.php'; // page générique
}
```

```
// app/Controller/AuthController.php - pas d'énumération de comptes (SEC-05)
$erreur = $identifiants['statut'] === 'COMPTE_BLOQUE'
    ? 'Ce compte est bloqué. Contactez le support.'
    : 'Email ou mot de passe erroné.'; // même message si l'email n'existe pas ou si le mot de
passe est faux
```

Preuve	E2E T2/T8 (« message générique », « ancien mot de passe désormais refusé ») ; les 5 suites exécutent des dizaines de refus métier sans jamais voir un détail interne. Le journal d'audit ne contient ni mot de passe ni hash (EF-GEN-04, vérifié T11)

S-09 - Force brute sur la connexion - ! écart assumé, correctif proposé

Risque	deviner un mot de passe à force de tentatives
Ce qui est en place	messages neutres identiques (pas d'énumération), `password_verify` (comparaison à temps constant), mots de passe hachés bcrypt (coût par tentative élevé), CSRF sur le formulaire
Ce qui manque	la **temporisation après 5 échecs** prévue par le document de tests (SEC-04/06) n'est **pas implémentée** : les tentatives échouent proprement et immédiatement. C'est l'écart le plus significatif de la livraison
Correctif proposé (à intégrer au lot L12)	compteur d'échecs par email en table dédiée, temporisation exponentielle côté serveur :

```
// Proposition - app/Security/Throttle.php (esquisse, ~25 lignes)
// après chaque échec : insert dans `tentative_connexion` (email, ip, at)
// avant chaque essai : $n = échecs des 15 dernières minutes
// if ($n >= 5) usleep(min($n - 4, 10) * 500_000);  // 0,25 s -> 1,25 s, plafonné
// succès ou changement de mot de passe : purge des lignes de l'email
```

À défaut de table, un `$_SESSION['echecs']` protège déjà par session (pas par compte) - la version table est la bonne.

S-10 - Élévation de privilège par le formulaire - [x] clés strictes, rôle hors de portée de la requête

Risque	`POST /inscription` avec `role=ADMIN` (ou `role=SUPER`) crée un administrateur
Attaque testée	E2E T12 : inscription d'« Ida » avec `role: 'SUPER'` glissé dans le POST ; T13 : un GESTIONNAIRE tente `POST /admin/equipe` pour créer un SUPER
Défense	le contrôleur ne lit **que** les clés attendues (`nom`, `prenom`, `email`, `mot_de_passe`, `confirmation`, `cgv`) - tout champ additionnel est ignoré ; le rôle d'un admin ne vient **jamais** de la requête mais de la table `administrateur`, et les gardes de routes s'appliquent **avant** le contrôleur :

```
// app/Router.php - la garde précède l'action : impossible de « oublier » de vérifier
public function executer(array $route): void
{
    match ($route['garde']) {
        'client' => Auth::exigeClient(),
        'admin'  => Auth::exigeAdmin(),
        'super'  => Auth::exigeSuper(), // 403 + « Réservé au rôle SUPER. »
        default  => null,
    };
    // ... puis seulement ici, l'action du contrôleur
}

// app/Security/Auth.php - le site public n'offre AUCUNE route créant un admin
public static function exigeSuper(): array
{
    $admin = self::exigeAdmin();
    if ($admin['role'] !== 'SUPER') {
        \http_response_code(403);
        $_SESSION['flash_erreur'] = 'Réservé au rôle SUPER.';
        self::rediriger('/admin');
    }
    return $admin;
}
```

Preuve	E2E T12 (« champ role=ADMIN ignoré », « le faux admin reste un simple client », table `administrateur` inchangée) + T13 (GESTIONNAIRE : back-office OK, `/admin/equipe` refusé, création d'admin impossible, SUPER peut désactiver un compte)

4. Mesures structurelles transverses

4.1 Mots de passe (RB-12, SEC-04)

• Jamais stockés en clair : `password_hash($mdp, PASSWORD_BCRYPT)` (vérifié en E2E : le hash persisté commence par `$2y$` et fait 60 caractères).

• Comparaison uniquement par `password_verify()` (temps constant) ; le repository **refuse** un

hash qui n'est pas une sortie de `password_hash()` (`HASH_MOT_DE_PASSE_INVALIDE`).

• Changement de mot de passe : ancien exigé et vérifié, confirmation requise (E2E T8 couvre : mauvais ancien refusé, ancien mot de passe révoqué après changement).

• Politique : 8 caractères minimum côté serveur (le `minlength` HTML n'est que du confort).

4.2 Sessions et espaces séparés (SEC-05, RB-13)

• Deux espaces **strictement** cloisonnés : `$_SESSION['client_id']` (front) et `$_SESSION['admin_id']` (back) ; un admin n'est pas un client et réciproquement (E2E T6).

• `session.use_strict_mode=1` : un identifiant forgé inconnu crée une session neuve, il n'est pas adopté.

• Régénération à chaque élévation/destruction de privilège (connexion, déconnexion).

• Durée de vie contrôlée par le moteur PHP ; le pool du bac à sable WASM est un artefact de **développement documenté** (en-tête de `Auth.php`), remplacé par l'API session native sur tout serveur réel.

4.3 Journal d'audit (EF-GEN-04, SEC-13)

• Toute écriture du moteur est tracée en JSONL (`var/journal/db.jsonl`) : `{at, op, table, pk, actor: {id, role, nom}, old, new}` - **qui** a fait **quoi**, avec avant/après.

• La piste des statuts de commande (RB-11) est requêtable et affichée au back-office (`order_status_history`, transition + auteur + commentaire).

• Les nouvelles entrées JSON de comptes client et administrateur sont limitées aux métadonnées de compte : les condensats, noms et e-mails ne figurent plus dans leurs instantanés `old/new`. Les anciennes entrées, les autres catégories, les noms d'acteurs administrateurs et les journaux ou sauvegardes externes doivent encore être examinés ; aucune suppression rétrospective n'a été effectuée.

4.4 Validation des entrées en profondeur (SEC-02)

• Accesseurs typés bornés dans le contrôleur de base : `$this->int('quantite')`, `$this->str('nom', 100)`, `$this->float('prix_min')`, `$this->bool('cgv')` - troncature et transtypage explicites **avant** tout usage.

• Puis contraintes du moteur (schéma) : types, NOT NULL, ENUM (`statut`), CHECK (`prix_ht > 0`, `stock >= 0`, `montant_total >= 0`), UNIQUE (`reference`, `slug`, `email`, `numero`), FK (RESTRICT/CASCADE) - testées par `engine_smoke.php` et `run_tests.php`.

• Les montants traversent `round(..., 2)` et les comparaisons d'égalité monétaire des tests tolèrent `0.001`.

4.5 Hygiène du code

• `declare(strict_types=1)` dans **100 % des fichiers PHP exécutables** (81/81 ; seul `app/Model/Data/Errors.php`, pure documentation, n'en a pas besoin) - vérifiable : `grep -rL "declare(strict_types=1)" app/ public/ tests/php/ scripts/`.

• **Zéro dépendance runtime** (pas de Composer dans l'application) : la surface d'attaque par dépendance (supply chain) est nulle.

• `app/Config/env.php` (identifiants) est non versionné (`.gitignore`), seul `env.example.php` est livré ; `debug` désactivé par défaut.

• Une classe par fichier, autoload PSR-4, pas d'`eval`, pas de `include` dynamique, pas de fonctions dangereuses (`system`, `exec`, `shell_exec`... - grep nul).

5. Traçabilité exigence -> code -> test

Exigence	Contre-mesure (code)	Preuve exécutée
S-01 / SEC-01/02	moteur JSON sans interpréteur, `Filter`, `matchesOne` (liste blanche), schéma à contraintes	E2E T12 (2), `features_test` (tri), `run_tests` T-25
S-02 / SEC-03	`e()` = `htmlspecialchars(ENT_QUOTES, UTF-8)` + CSP	E2E T12 (2)
S-03 / SEC-08	contrôle d'appartenance (`ACCES_NON_AUTORISE`)	E2E T12 (boucle IDOR), T6, `features_test`
S-04 / SEC-10	prix recalculé serveur (`PanierSession::recapitulatif`), figé (RB-06)	E2E T9, `run_tests`
S-05 / SEC-05	cookie durci + `regenererId()` à l'élévation	E2E T12 (2)
S-06 / SEC-06	`Csrf::verifier()` (`hash_equals`, 403) sur **tout** POST	E2E T6, `unit_tests`
S-07 / SEC-11	routeur liste blanche, `DocumentRoot=public/`, `.htaccess`, 5 en-têtes	E2E T12 (8 chemins + 4 en-têtes), T1
S-08 / SEC-12	`messageHumain()` des erreurs métier ; `Throwable` -> 500 générique + `error_log` ; messages d'auth unifiés	E2E T2/T8 + 5 suites
S-09 / SEC-04/06	messages neutres + bcrypt + temps constant - **temporisation : non implémentée (§3)**	E2E T2 (messages) - écart documenté
S-10 / SEC-10 / RB-13	gardes de route avant contrôleur, clés strictes, rôle hors requête	E2E T12 (3), T13 (6), T6
SEC-13 / EF-GEN-04	journal JSONL horodaté avec acteur	E2E T11 (4 vérifications)
RB-12	bcrypt + `password_verify` + hash non-sortie-refusé	E2E T2/T8, `features_test`

6. Limites assumées et pistes de durcissement

#	Limite	Statut / proposition
1	**Anti-force-brute** (temporisation après 5 échecs)	non implémenté - correctif esquisssé en §3-S-09 (compteur + temporisation exponentielle) ; à intégrer au lot L12

2	`x-powered-by` du bac à sable WASM	artefact du runtime de dev ; en production : `expose_php=Off`, `ServerTokens Prod` (config serveur, hors application)
3	HTTPS/TLS, HSTS	hors périmètre applicatif (reverse proxy) ; le cookie passe `secure` dès que `HTTPS` est détecté
4	En-tête `Permissions-Policy`	prévu par le CDC (SEC-11), pas encore émis - une ligne à ajouter dans `public/index.php`
5	Journal `old/new` lors d'un changement de mot de passe	le hash bcrypt transite dans l'entrée de journal (il est déjà haché, jamais en clair) ; à masquer si la politique de rétention l'exige
6	Clé CSRF par session (pas par formulaire)	standard et suffisant avec `SameSite=Lax` ; une rotation par formulaire renforcerait contre la fixation de jeton
7	Expiration/rotation automatique des sessions	déléguée au moteur PHP ; à borner explicitement (`session.gc_maxlifetime`) en production
8	Images = URL externes	pas d'upload (pas de surface) ; si un upload est ajouté un jour : type MIME + extension + taille + stockage hors `DocumentRoot` (SEC-11)
9	Outils de volumétrie/mesure (`public/gen_volumes.php`, `mesurer.php`)	hérités de la phase SQL : ils visent la base MySQL de référence, pas le moteur JSON ; verrou = `APP_ENV` (404 en prod). À re-brancher sur le moteur JSON ou à retirer de `public/` au lot ENF-01

7. Annexe - rejouer les attaques

```
npm run test:all          # 5 suites : moteur, règles 29/29, unitaires 98/98,
                          # fonctionnalités 108/108, E2E 155/155
                          # (T6+T12+T13 = 45 vérifications de sécurité)
```

Attaques rejouables individuellement (extraits de `scripts/wasm-e2e.mjs`) :

```
// S-01  GET /recherche?q=' OR 1=1 --            -> 200, « 0 produit trouvé »
// S-02  produit « <script>alert("xss")</script> » -> affiché « &lt;script&gt; »
// S-03  boucle /mes-commandes/1..4 en Bruno      -> 302 systématique
// S-04  POST /panier/ajouter {prix_unitaire:0.01} -> total au vrai prix serveur
// S-05  cookie imposé puis connexion             -> sid régénéré, panier vide
// S-06  POST /connexion sans _csrf               -> 403 + redirection
// S-07  /.git/config, /sql/..., /../etc/passwd     -> 404 ; 4 en-têtes vérifiés
// S-10  inscription avec role=SUPER              -> client simple, table admin intacte
```

Note rédigée le 28/09/2026 - toutes les affirmations ci-dessus sont vérifiables dans le dépôt : les extraits de code sont cités fidèlement, les tests sont exécutables par `npm run test:all`.